15.4 超时与中止机制

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~350,000 tokens，输出 ~28,000 tokens（本章合计）

LLM 调用天生具有不确定性——模型可能因速率限制而挂起，复杂的工具调用链可能无限循环，或者用户可能改变主意想取消正在进行的操作。OpenClaw 通过多层超时和中止机制来应对这些场景，确保系统始终保持响应能力。

15.4.1 Agent 运行超时（默认 600 秒）

每次 Agent 运行都有一个最大执行时间限制。超过这个时间，运行会被强制终止。

超时解析

```
// src/agents/timeout.ts
const DEFAULT_AGENT_TIMEOUT_SECONDS = 600; // 10 分钟

export function resolveAgentTimeoutSeconds(cfg?: OpenClawConfig): number {
  const raw = normalizeNumber(cfg?.agents?.defaults?.timeoutSeconds);
  const seconds = raw ?? DEFAULT_AGENT_TIMEOUT_SECONDS;
  return Math.max(seconds, 1);  // 最小 1 秒
}

export function resolveAgentTimeoutMs(opts: {
  cfg?: OpenClawConfig;
  overrideMs?: number | null;
  overrideSeconds?: number | null;
  minMs?: number;
}): number {
  const minMs = Math.max(normalizeNumber(opts.minMs) ?? 1, 1);
  const defaultMs = resolveAgentTimeoutSeconds(opts.cfg) * 1000;

  // 特殊值: 0 表示"无超时" (实际设为 30 天)
  const NO_TIMEOUT_MS = 30 * 24 * 60 * 60 * 1000;
  const overrideSeconds = normalizeNumber(opts.overrideSeconds);
  if (overrideSeconds === 0) return NO_TIMEOUT_MS;
  if (overrideSeconds && overrideSeconds > 0) {
    return Math.max(overrideSeconds * 1000, minMs);
  }
  return Math.max(defaultMs, minMs);
}
```

超时配置有三个来源，优先级从高到低：

来源	配置路径	说明
请求参数	`agent` RPC 的 `timeout` 字段	单次调用的超时覆盖
全局配置	`agents.defaults.timeoutSeconds`	所有 Agent 的默认超时
硬编码默认	600 秒	无配置时的回退值

将超时设为 `0` 是一个特殊值，表示“实际上不超时”。由于 JavaScript 的 `setTimeout` 不支持 `Infinity`，系统用 30 天（约 260 万秒）作为替代。

15.4.2 `agent.wait` 等待超时（默认 30 秒）

`agent.wait` 是 Gateway 提供的一个 RPC 方法，允许调用方阻塞等待某个 Agent 运行完成。它主要被子 Agent 注册表用来等待子 Agent 的完成。

```
// src/gateway/server-methods/agent.ts
"agent.wait": async ({ params, respond }) => {
  const runId = params.runId.trim();
  const timeoutMs = typeof params.timeoutMs === "number" && Number.isFinite(params.time
    ? Math.max(0, Math.floor(params.timeoutMs))
    : 30_000;  // 默认 30 秒

  const snapshot = await waitForAgentJob({ runId, timeoutMs });
  if (!snapshot) {
    respond(true, { runId, status: "timeout" });
    return;
  }
  respond(true, {
    runId,
    status: snapshot.status,
    startedAt: snapshot.startedAt,
    endedAt: snapshot.endedAt,
    error: snapshot.error,
  });
};
```

等待机制的内部实现

`waitForAgentJob` 用了一个基于事件监听的等待模式：

```ts
// src/gateway/server-methods/agent-job.ts
export async function waitForAgentJob(params: {
  runId: string;
  timeoutMs: number;
}): Promise<AgentRunSnapshot | null> {
  // 1. 先检查缓存（运行可能已经结束）
  const cached = getCachedAgentRun(params.runId);
  if (cached) return cached;

  // 2. 订阅生命周期事件并等待
  return await new Promise((resolve) => {
    let settled = false;
    const finish = (entry: AgentRunSnapshot | null) => {
      if (settled) return;
      settled = true;
      clearTimeout(timer);
      unsubscribe();
      resolve(entry);
    };

    // 监听 Agent 运行的 end/error 事件
    const unsubscribe = onAgentEvent((evt) => {
      if (evt?.stream !== "lifecycle" || evt.runId !== params.runId) return;
      if (evt.data?.phase !== "end" && evt.data?.phase !== "error") return;
      finish(getCachedAgentRun(params.runId) ?? buildSnapshot(evt));
    });

    // 超时回退
    const timer = setTimeout(() => finish(null), Math.max(1, params.timeoutMs));
  });
}
```

这个实现有几个值得注意的设计决策：

1. **先查缓存再等待**——避免竞态条件：如果运行在创建 Promise 之前就已完成，缓存查询可以立即返回
2. **settled 标志保证单次 resolve**—— `finish` 函数只会执行一次，无论是事件触发还是超时触发
3. **自动清理**——无论哪个路径触发了 resolve，都会同时清理定时器和事件订阅

运行快照缓存

完成的 Agent 运行会在缓存中保留 10 分钟：

```typescript
// src/gateway/server-methods/agent-job.ts
const AGENT_RUN_CACHE_TTL_MS = 10 * 60_000;
const agentRunCache = new Map<string, AgentRunSnapshot>();

type AgentRunSnapshot = {
  runId: string;
  status: "ok" | "error";
  startedAt?: number;
  endedAt?: number;
  error?: string;
  ts: number;
};
```

10 分钟的 TTL 保证了：即使 `agent.wait` 调用略晚于 Agent 运行完成，它仍能从缓存中获取结果，而不会因为错过事件而永久阻塞。

15.4.3 AbortSignal 取消链

JavaScript 的 `AbortSignal` / `AbortController` 机制为 OpenClaw 提供了一种贯穿整个调用栈的**协作式取消**方案。

> **衍生解释**： `AbortController` 和 `AbortSignal` 是 Web API 中的标准机制，已被 Node.js 完整实现。 `AbortController` 创建一个控制器，它持有一个 `AbortSignal` 对象。当调用 `controller.abort()` 时， `signal` 会触发 `abort` 事件，所有监听该 signal 的操作都会被通知取消。这类似于 Go 语言中的 `context.Context` 或 C# 中的 `CancellationToken` 。

取消链的传播路径

```
用户发送 /stop 命令
    │
    ▼
Gateway 查找对应的 AbortController
    │
    ▼
controller.abort()
    │
    ▼
AbortSignal 传播到:
    ├── LLM HTTP 请求（fetch 的 signal 参数）
    ├── 工具执行（bash 进程的 kill）
    ├── Pi Agent 对话循环（检测 signal.aborted）
    └── 流式输出（停止发送 delta 事件）
```

在 `runEmbeddedPiAgent` 中， `abortSignal` 作为参数传递到每一层：

```
// src/agents/pi-embedded-runner/run.ts（参数传递链）
const attempt = await runEmbeddedAttempt({
  // ...其他参数
  abortSignal: params.abortSignal,  // 传入 AbortSignal
});
```

当 signal 被触发时，正在运行的 LLM 请求会被取消（通过 `fetch` 的 signal 支持），工具执行会被中断，整个 Agent 循环会在下一个检查点退出。

活跃运行中止

每个活跃的 Agent 运行都注册了一个中止句柄：

```
// src/agents/pi-embedded-runner/runs.ts
export function abortEmbeddedPiRun(sessionId: string): boolean {
  const handle = ACTIVE_EMBEDDED_RUNS.get(sessionId);
  if (!handle) return false;
  handle.abort();
  return true;
}
```

`abort()` 调用会：

1. 触发 AbortSignal 的 abort 事件
2. 取消正在进行的 HTTP 请求
3. 将运行标记为 `aborted`（但不立即删除——等清理完成后删除）

15.4.4 聊天中止（`src/gateway/chat-abort.ts`）

Gateway 层面的中止管理比单个 Agent 运行更复杂，因为它需要处理多运行、跨连接的场景。

中止控制器注册

每个 Agent 运行在 Gateway 中都有一个对应的中止控制器条目：

```
// src/gateway/chat-abort.ts
export type ChatAbortControllerEntry = {
  controller: AbortController;
  sessionId: string;
  sessionKey: string;
  startedAtMs: number;
  expiresAtMs: number;  // 过期时间（即使忘记取消也会自动清理）
};
```

`expiresAtMs` 是一个安全网——即使中止逻辑出错导致运行没有被正常清理，过期时间也会保证资源最终被回收。过期时间的计算考虑了运行超时加上一个宽限期：

```
// src/gateway/chat-abort.ts
export function resolveChatRunExpiresAtMs(params: {
  now: number;
  timeoutMs: number;
  graceMs?: number;   // 默认 60 秒宽限
  minMs?: number;     // 默认 2 分钟最小值
  maxMs?: number;     // 默认 24 小时最大值
}): number {
  const target = params.now + params.timeoutMs + (params.graceMs ?? 60_000);
  const min = params.now + (params.minMs ?? 2 * 60_000);
  const max = params.now + (params.maxMs ?? 24 * 60 * 60_000);
  return Math.min(max, Math.max(min, target));
}
```

停止命令检测

用户可以通过发送 `/stop` 命令来中止当前正在运行的 Agent：

```
// src/gateway/chat-abort.ts
export function isChatStopCommandText(text: string): boolean {
  const trimmed = text.trim();
  if (!trimmed) return false;
  return trimmed.toLowerCase() === "/stop" || isAbortTrigger(trimmed);
}
```

除了 `/stop` 外， `isAbortTrigger` 还支持其他触发模式（如自定义配置的中止关键词）。

按运行 ID 中止

```
// src/gateway/chat-abort.ts
export function abortChatRunById(
  ops: ChatAbortOps,
  params: { runId: string; sessionKey: string; stopReason?: string },
): { aborted: boolean } {
  const active = ops.chatAbortControllers.get(params.runId);
  if (!active || active.sessionKey !== params.sessionKey) return { aborted: false };

  // 1. 记录中止事件
  ops.chatAbortedRuns.set(params.runId, Date.now());
  // 2. 触发 AbortController
  active.controller.abort();
  // 3. 清理资源
  ops.chatAbortControllers.delete(params.runId);
  ops.chatRunBuffers.delete(params.runId);
  ops.chatDeltaSentAt.delete(params.runId);
  ops.removeChatRun(params.runId, params.runId, params.sessionKey);
  // 4. 广播中止事件
  broadcastChatAborted(ops, { runId: params.runId, sessionKey: params.sessionKey, stopR
  return { aborted: true };
}
```

中止操作的四个步骤是原子性的——一旦开始中止，所有相关状态都会被清理，中止事件会被广播给所有连接的客户端。

按会话键批量中止

当用户发送 `/stop` 时，需要中止该会话中所有正在运行的 Agent：

```
// src/gateway/chat-abort.ts
export function abortChatRunsForSessionKey(
  ops: ChatAbortOps,
  params: { sessionKey: string; stopReason?: string },
): { aborted: boolean; runIds: string[] } {
  const runIds: string[] = [];
  for (const [runId, active] of ops.chatAbortControllers) {
    if (active.sessionKey !== params.sessionKey) continue;
    const res = abortChatRunById(ops, { runId, sessionKey: params.sessionKey, stopReaso
    if (res.aborted) runIds.push(runId);
  }
  return { aborted: runIds.length > 0, runIds };
}
```

这个函数遍历所有活跃的中止控制器，找到属于目标会话的所有运行并逐一中止。

中止事件广播

中止操作会通过 WebSocket 广播一个 `chat` 事件，通知所有客户端：

```
function broadcastChatAborted(ops: ChatAbortOps, params: {
  runId: string;
  sessionKey: string;
  stopReason?: string;
}) {
  const payload = {
    runId: params.runId,
    sessionKey: params.sessionKey,
    seq: (ops.agentRunSeq.get(params.runId) ?? 0) + 1,
    state: "aborted" as const,
    stopReason: params.stopReason,
  };
  ops.broadcast("chat", payload);                          // 广播给所有客户端
  ops.nodeSendToSession(params.sessionKey, "chat", payload); // 也发送到会话节点
}
```

客户端收到中止事件后可以：

- 停止显示"正在输入..."指示器
- 在消息气泡上标记"已取消"
- 清理流式输出缓冲区

多层超时与中止的完整图景

下表总结了 OpenClaw 中所有的超时和中止机制：

层级	机制	默认值	触发方式
LLM 请求	HTTP 请求超时	由提供者 SDK 控制	网络超时自动触发
Agent 运行	`resolveAgentTimeoutMs`	600 秒	计时器自动触发
Agent 等待	`agent.wait`	30 秒	等待超时返回 null
用户中止	`/stop` 命令	无	用户手动触发
资源清理	`expiresAtMs`	超时 + 60 秒宽限	过期自动清理
AbortSignal	协作式取消	无	由上层触发传播

这些机制层层嵌套，形成了一个完整的安全网——即使某一层的超时/中止失败，上层机制也会最终保证资源被回收。

本节小结

1. **Agent 运行超时**默认 600 秒，可通过配置文件或请求参数覆盖。特殊值 `0` 表示不超时（实际为 30 天）。
2. **`agent.wait` RPC 方法**允许调用方阻塞等待 Agent 运行完成，使用事件监听 + 超时的 Promise 模式实现，运行快照缓存 10 分钟防止竞态条件。
3. **AbortSignal 取消链**贯穿整个调用栈，从 Gateway 到 LLM HTTP 请求，实现了协作式取消。
4. **Gateway 级中止管理**包括按运行 ID 中止和按会话批量中止，中止操作原子性地清理所有相关状态并广播事件通知客户端。
5. **多层安全网**确保即使某一层机制失败，系统也不会泄漏资源——过期时间机制是最后的保障。